

# Secure DMS — Implementation Specification

Role-Based Auth · Case-Centric Data Model · Live Supabase Backend · Judge Demo Script

**Problem Statement 26190** — Ministry of Home Affairs / NCRB (Women Safety Division) **Theme:** Blockchain & Cybersecurity · **Category:** Software

This is a build document, not a concept note. Every table, policy and function below is meant to be pasted into a real Supabase project and run. By the end of it you have a working app with real accounts, real OTP, real persisted data, and live cross-device updates.

## 0. The one decision that makes this "not a prototype"

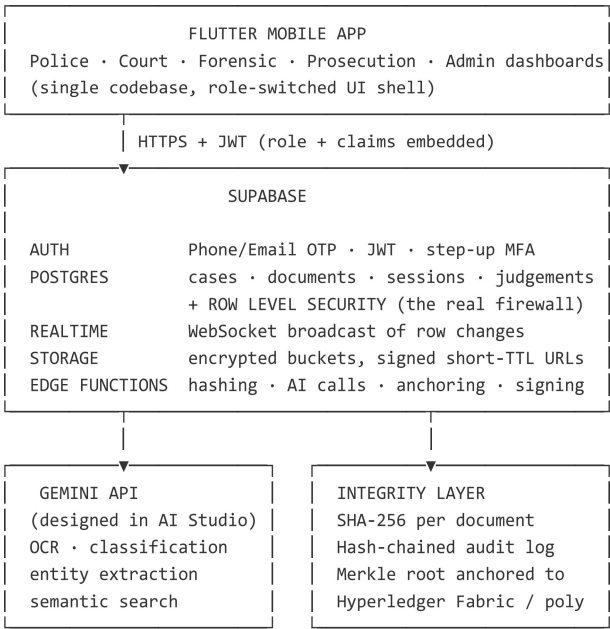
Most hackathon DMS projects fail the judge's real test, which is some version of:

*"Is this actually storing anything, or is it a UI over a JSON file?"*

The answer has to be provable on stage. So the architecture is chosen so that **the database enforces the security, not the app**. If a judge takes your phone, logs in as a constable, and tries to open another district's case, the *server* refuses — not a hidden `if` statement in your Dart code. That is the difference between a demo and a system.

Concretely: **Postgres Row-Level Security (RLS) is the security boundary**. Supabase gives you Postgres + Auth + Realtime + Storage on top of it, so the same policy that blocks the API call also blocks the realtime subscription and the file download. One rule, three enforcement points.

## 1. System architecture



**Why Supabase over a hand-rolled Node + Mongo backend:** you get authentication, a relational database, row-level authorization, file storage and a realtime socket layer that are *already wired to each other*. In a hackathon timeline, hand-rolling those four integrations is where teams lose two days and end up faking the realtime part. It is also plain Postgres underneath — so nothing here is locked in, and you can answer "can this run on NIC/on-prem government infrastructure?" with "yes, Supabase self-hosts; it's Postgres."

## 2. Identity model — who is allowed to exist

### 2.1 The problem with plain signup

Anyone can type "I am a judge" into a registration form. In a police/judiciary system that is unacceptable. So registration is **two-phase**:

- 1. **Self-registration** → user signs up with official phone/email, submits badge/service number, designation, and posting (station or court). Account is created with `status = 'pending'`.
- 2. **Departmental approval** → an Admin (or the department head role) verifies against the service record and flips them to `approved`. Only then do RLS policies start returning data.

A pending user can log in and see exactly one screen: *"Awaiting verification by your department administrator."* Nothing else in the database is visible to them.

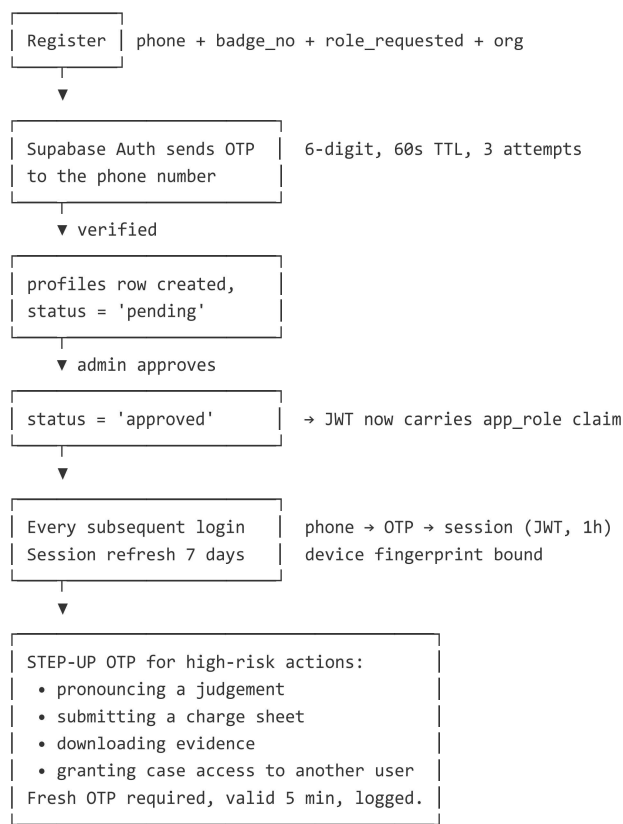
This is not extra friction for the sake of it — it's the answer to the judge question *"how do you prevent impersonation of a judicial officer?"*

2.2 Roles

| Role                  | Can do                                                                            |
|-----------------------|-----------------------------------------------------------------------------------|
| admin                 | Approve users, manage stations/courts, read audit log, cannot read case documents |
| police_officer        | Create cases (FIR), upload investigation docs to own cases                        |
| investigating_officer | Full document control on assigned cases, submit charge sheet                      |
| prosecutor            | Read all docs on cases filed to their court, file court submissions               |
| judge                 | Read case bundle, create court sessions, pronounce judgement, lock documents      |
| court_clerk           | Schedule sessions, upload court filings, no judgement rights                      |
| forensic_expert       | Upload forensic reports, read only evidence docs on assigned cases                |

**Deliberate design choice worth defending:** `admin` has user-management power but **no document read access**. A system administrator should never be able to read a witness statement. Judges love this — it shows you thought about insider threat, not just outsider threat.

2.3 Authentication flow (OTP, real)



**Step-up OTP is your differentiator here.** Every app has login OTP. Almost none re-authenticate at the moment of a legally consequential action. In a courtroom context — where a signed judgement is binding — proving the human

was physically present at the moment of signing matters. Say exactly this to the judges.

For the demo, use **email OTP** rather than SMS. Twilio trial accounts throttle and will embarrass you on stage.

Supabase email OTP is free, instant, and works on a venue wifi. Mention SMS/DSC integration as the production path.

### 3. Database schema

Run this in the Supabase SQL editor. It is the complete schema.

#### 3.1 Extensions and enums

```
create extension if not exists pgcrypto; -- digest(), gen_random_uuid()
create extension if not exists pg_trgm; -- fuzzy text search

create type app_role as enum (
  'admin', 'police_officer', 'investigating_officer',
  'prosecutor', 'judge', 'court_clerk', 'forensic_expert'
);

create type account_status as enum ('pending', 'approved', 'suspended', 'revoked');

create type case_stage as enum (
  'fir_registered', 'under_investigation', 'charge_sheet_filed',
  'in_trial', 'judgement_reserved', 'disposed', 'appealed', 'closed'
);

create type doc_type as enum (
  'fir', 'police_report', 'investigation_record', 'witness_statement',
  'charge_sheet', 'court_filing', 'evidence_record', 'forensic_report',
  'legal_notice', 'judgement', 'other'
);

create type doc_status as enum ('draft', 'submitted', 'verified', 'locked', 'superseded');
```

Note that `doc_type` maps **one-to-one onto the eight document categories in the problem statement**. Show this enum to the judges. It is a thirty-second way to prove you read the requirement rather than building a generic file drive.

#### 3.2 Organisations and profiles

```
create table organisations (
  id          uuid primary key default gen_random_uuid(),
  org_type    text not null check (org_type in ('police_station', 'court', 'forensic_lab', 'prosecution_office')),
  name        text not null,
  district    text not null,
  state       text not null,
  code        text unique not null,          -- e.g. 'UP-LKO-PS-HZ'
  created_at  timestampz not null default now()
);

create table profiles (
  id          uuid primary key references auth.users(id) on delete cascade,
  full_name   text not null,
  badge_no    text not null,
  designation  text not null,
  role        app_role not null,
  org_id      uuid not null references organisations(id),
  phone       text,
  email       text,
  status      account_status not null default 'pending',
  approved_by uuid references profiles(id),
  approved_at timestampz,
  created_at  timestampz not null default now(),
  unique (badge_no, org_id)
);
```

#### 3.3 Cases

```

create table cases (
  id          uuid primary key default gen_random_uuid(),
  case_number text unique not null,      -- 'CASE-2026-000001'
  fir_number  text,                     -- 'FIR/142/2026'
  title       text not null,
  sections    text[],                   -- BNS/IPC sections invoked
  stage       case_stage not null default 'fir_registered',
  filed_by    uuid not null references profiles(id),
  police_org_id uuid not null references organisations(id),
  court_org_id uuid references organisations(id),
  district    text not null,
  is_sensitive boolean not null default false, -- POCSO / women safety: tighter ACL
  created_at  timestamptz not null default now(),
  updated_at  timestamptz not null default now()
);

-- Human-readable sequential case numbers
create sequence case_seq start 1;
create or replace function gen_case_number() returns trigger
language plpgsql as $$
begin
  if new.case_number is null then
    new.case_number := 'CASE-' || to_char(now(),'YYYY') || '-' ||
      lpad(nextval('case_seq')::text, 6, '0');
  end if;
  return new;
end $$;

create trigger trg_case_number before insert on cases
for each row execute function gen_case_number();

```

### 3.4 Case access control (ABAC layer)

Role alone is not enough. A judge in Lucknow should not read a case in Kanpur. Access is granted **per case, per person**.

```

create table case_participants (
  id          uuid primary key default gen_random_uuid(),
  case_id     uuid not null references cases(id) on delete cascade,
  user_id     uuid not null references profiles(id) on delete cascade,
  role_in_case text not null,      -- 'io','supervising_officer','prosecutor','presiding_judge','forensic'
  granted_by  uuid references profiles(id),
  granted_at  timestamptz not null default now(),
  revoked_at  timestamptz,
  unique (case_id, user_id)
);

```

When a police officer files a case, they are auto-added as `io`. When the case is committed to a court, every approved `judge / court_clerk` of that court org gets a participant row — handled by a trigger so it's automatic, not a manual demo step:

```

create or replace function auto_enroll_creator() returns trigger
language plpgsql security definer as $$
begin
  insert into case_participants (case_id, user_id, role_in_case, granted_by)
  values (new.id, new.filed_by, 'io', new.filed_by);
  return new;
end $$;

create trigger trg_enroll_creator after insert on cases
for each row execute function auto_enroll_creator();

```

### 3.5 Documents and versioning

```

create table documents (
  id          uuid primary key default gen_random_uuid(),
  case_id     uuid not null references cases(id) on delete restrict,
  . . .

```

```

doc_type      doc_type not null,
title         text not null,
storage_path  text not null,          -- bucket path
mime_type     text,
size_bytes    bigint,
sha256        text not null,         -- computed client-side, re-verified server-side
version       int not null default 1,
supersedes    uuid references documents(id), -- version chain
is_current     boolean not null default true,
status        doc_status not null default 'submitted',
uploaded_by   uuid not null references profiles(id),
ocr_text      text,                  -- Gemini OCR output
ai_entities    jsonb,                -- extracted names, dates, sections
ai_summary     text,
created_at    timestampz not null default now()
);

create index on documents (case_id, doc_type);
create index on documents using gin (to_tsvector('english', coalesce(ocr_text,'')));
create index on documents using gin (title gin_trgm_ops);

```

**Version control done properly:** documents are immutable. An "edit" inserts a new row with `version = old.version + 1` and `supersedes = old.id`, then sets the old row's `is_current = false`. Nothing is ever overwritten, so the tamper question has a structural answer: there is no UPDATE path on document content to exploit.

```

-- Block content mutation outright
create or replace function documents_immutable() returns trigger
language plpgsql as $$
begin
  if old.sha256 is distinct from new.sha256
    or old.storage_path is distinct from new.storage_path then
    raise exception 'Document content is immutable. Create a new version instead.';
  end if;
  if old.status = 'locked' and new.status <> 'locked' then
    raise exception 'Locked documents cannot be unlocked.';
  end if;
  return new;
end $$;

create trigger trg_doc_immutable before update on documents
for each row execute function documents_immutable();

```

### 3.6 Court sessions and judgements — the live-update core

This is the part the user flow depends on: *police files case* → *court records session* → *judgement appears under that case with timestamp*.

```

create table court_sessions (
  id              uuid primary key default gen_random_uuid(),
  case_id         uuid not null references cases(id) on delete cascade,
  court_org_id    uuid not null references organisations(id),
  hearing_type    text not null, -- 'first_hearing', 'framing_of_charges', 'evidence', 'arguments', 'pronour
  scheduled_at    timestampz not null,
  held_at         timestampz,
  presiding_judge uuid references profiles(id),
  proceedings     text,          -- what happened in court
  next_hearing_at timestampz,
  recorded_by     uuid not null references profiles(id),
  created_at      timestampz not null default now()
);

create table judgements (
  id              uuid primary key default gen_random_uuid(),
  case_id         uuid not null references cases(id) on delete restrict,
  session_id      uuid references court_sessions(id),
  verdict         text not null, -- 'convicted', 'acquitted', 'discharged', 'settled', 'remanded'
  operative_order text not null,
  full_text       text,
  document_id     uuid references documents(id),
  pronounced_by   uuid not null references profiles(id),
  pronounced_at   timestampz not null default now(),

```

```

sha256      text not null,
signature   text,          -- detached digital signature (DSC/eSign in prod)
created_at  timestampz not null default now()
);

```

A judgement automatically advances the case and locks the evidence — enforced in the database, so it's true no matter which client writes it:

```

create or replace function on_judgement_pronounced() returns trigger
language plpgsql security definer as $$
begin
    update cases
        set stage = 'disposed', updated_at = now()
        where id = new.case_id;

    update documents
        set status = 'locked'
        where case_id = new.case_id and is_current = true;

    return new;
end $$;

create trigger trg_judgement after insert on judgements
for each row execute function on_judgement_pronounced();

```

**Say this to a judge:** "Once a verdict is pronounced, every document on that case becomes immutable at the database level. Not by convention — by trigger. Even a developer with API access cannot alter the evidentiary record after disposal."

#### 4. Integrity: the hash-chained audit log

Every action writes an audit row, and each row's hash includes the previous row's hash. Deleting or editing any single log entry breaks every hash after it.

```

create table audit_log (
    id          bigserial primary key,
    actor_id    uuid references profiles(id),
    action      text not null,          -- 'CASE_CREATE', 'DOC_UPLOAD', 'DOC_VIEW', 'JUDGEMENT_PRONOUNCE', 'ACCESS_
    entity_type text not null,
    entity_id   uuid,
    case_id     uuid references cases(id),
    metadata    jsonb default '{} '::jsonb,
    ip_address  inet,
    device_id   text,
    prev_hash   text,
    record_hash text not null,
    created_at  timestampz not null default now()
);

create or replace function audit_chain() returns trigger
language plpgsql as $$
declare
    last_hash text;
    payload   text;
begin
    perform pg_advisory_xact_lock(778899); -- serialise chain writes

    select record_hash into last_hash
        from audit_log order by id desc limit 1;

    new.prev_hash := coalesce(last_hash, repeat('0', 64));

    payload := coalesce(new.actor_id::text, '') || '|' || new.action || '|' ||
        new.entity_type || '|' || coalesce(new.entity_id::text, '') || '|' ||
        coalesce(new.metadata::text, '{}') || '|' ||
        new.created_at::text || '|' || new.prev_hash;

```

```

new.record_nasn := encode(digest(payload, 'sha256'), 'hex');
return new;
end $$;

create trigger trg_audit_chain before insert on audit_log
for each row execute function audit_chain();

-- Nothing may ever modify or delete an audit row
create rule audit_no_update as on update to audit_log do instead nothing;
create rule audit_no_delete as on delete to audit_log do instead nothing;

```

Verification function you can run live on stage:

```

create or replace function verify_audit_chain()
returns table(broken_at bigint, expected text, found text)
language plpgsql as $$
declare
  r record; prev text := repeat('0',64); calc text;
begin
  for r in select * from audit_log order by id loop
    calc := encode(digest(
      coalesce(r.actor_id::text,'') || '|' || r.action || '|' ||
      r.entity_type || '|' || coalesce(r.entity_id::text,'') || '|' ||
      coalesce(r.metadata::text,'{}') || '|' || r.created_at::text || '|' || prev
      , 'sha256'), 'hex');
    if calc <> r.record_hash then
      broken_at := r.id; expected := calc; found := r.record_hash; return next;
    end if;
    prev := r.record_hash;
  end loop;
end $$;

```

### Blockchain anchoring (the theme requirement, done honestly)

Writing every action to a blockchain is slow and pointless. Instead: **batch anchoring**. Every 15 minutes (or every N events), an Edge Function takes the latest `record_hash`, builds a Merkle root over the batch, and writes that single root to a permissioned Hyperledger Fabric channel.

```

create table chain_anchors (
  id          uuid primary key default gen_random_uuid(),
  from_log_id bigint not null,
  to_log_id   bigint not null,
  merkle_root text not null,
  chain_tx_id text,
  chain_name  text default 'hyperledger-fabric',
  anchored_at timestampz not null default now()
);

```

**Defend it like this:** the hash chain gives you tamper *detection* within the database. The blockchain anchor gives you tamper *evidence that survives a compromised database administrator* — because the root is held by parties who don't run your server. That's the entire reason for the chain, and it's a one-sentence answer to "why not just a locked-down database?"

## 5. Row-Level Security — the actual firewall

Helper functions first (security definer so they can read profiles without recursion):

```

create or replace function current_role_of() returns app_role
language sql stable security definer as $$
  select role from profiles where id = auth.uid() and status = 'approved'
$$;

create or replace function is_approved() returns boolean
language sql stable security definer as $$
  select exists (select 1 from profiles where id = auth.uid() and status = 'approved')
$$;

```

```

create or replace function is_case_member(cid uuid) returns boolean
language sql stable security definer as $$
  select exists (
    select 1 from case_participants cp
    join profiles p on p.id = cp.user_id
    where cp.case_id = cid
      and cp.user_id = auth.uid()
      and cp.revoked_at is null
      and p.status = 'approved'
  )
  $$;

```

Enable and lock down:

```

alter table profiles          enable row level security;
alter table cases             enable row level security;
alter table case_participants enable row level security;
alter table documents         enable row level security;
alter table court_sessions    enable row level security;
alter table judgements        enable row level security;
alter table audit_log         enable row level security;

```

## Profiles

```

create policy "read own profile" on profiles
for select using (id = auth.uid());

create policy "admin reads all profiles" on profiles
for select using (current_role_of() = 'admin');

create policy "create own profile on signup" on profiles
for insert with check (id = auth.uid() and status = 'pending');

create policy "admin approves" on profiles
for update using (current_role_of() = 'admin');

```

## Cases — participation-based, not role-based

```

create policy "members read case" on cases
for select using (is_approved() and is_case_member(id));

create policy "police file case" on cases
for insert with check (
  is_approved()
  and current_role_of() in ('police_officer', 'investigating_officer')
  and filed_by = auth.uid()
);

create policy "io updates own case" on cases
for update using (
  is_case_member(id)
  and current_role_of() in ('investigating_officer', 'police_officer', 'judge')
);

```

## Documents — read requires case membership; forensic sees only forensic

```

create policy "members read docs" on documents
for select using (
  is_case_member(case_id)
  and (
    current_role_of() <> 'forensic_expert'
    or doc_type in ('forensic_report', 'evidence_record')
  )
);

create policy "members upload docs" on documents
for insert with check (
  is_case_member(case_id)
  and uploaded_by = auth.uid()
);

```

```

    and current_role_of() <> 'admin'
  );

```

### Judgements — only a judge, only on their own case

```

create policy "members read judgements" on judgements
for select using (is_case_member(case_id));

create policy "only judge pronounces" on judgements
for insert with check (
  current_role_of() = 'judge'
  and is_case_member(case_id)
  and pronounced_by = auth.uid()
);

```

### Court sessions

```

create policy "members read sessions" on court_sessions
for select using (is_case_member(case_id));

create policy "court staff record sessions" on court_sessions
for insert with check (
  current_role_of() in ('judge', 'court_clerk')
  and is_case_member(case_id)
  and recorded_by = auth.uid()
);

```

### Audit log — visible, never writable from the client

```

create policy "members read case audit" on audit_log
for select using (
  (case_id is not null and is_case_member(case_id))
  or current_role_of() = 'admin'
);
-- no insert policy: only Edge Functions (service role) may write

```

This block of SQL is your single strongest demo asset. Project it on screen when a judge asks about security. Most teams answer with a paragraph; you answer with enforced policy.

## 6. Storage and file security

```

-- Bucket: case-documents (private)
insert into storage.buckets (id, name, public)
values ('case-documents', 'case-documents', false);

create policy "read own case files" on storage.objects
for select using (
  bucket_id = 'case-documents'
  and is_case_member( (storage.foldername(name))[1]::uuid )
);

create policy "write own case files" on storage.objects
for insert with check (
  bucket_id = 'case-documents'
  and is_case_member( (storage.foldername(name))[1]::uuid )
);

```

Path convention: case-documents/{case\_id}/{doc\_type}/{uuid}.{ext} — so the folder name is the authorization key.

Downloads use signed URLs with a 60-second TTL, generated per request and logged. No permanent public link to a witness statement ever exists.

Upload sequence (client):

```

pick file → compute SHA-256 locally → upload to storage
→ insert documents row with hash

```

- Edge Function re-downloads, re-hashes, compares
- mismatch ⇒ row deleted, AUDIT: DOC\_HASH\_MISMATCH raised

Hashing **before** upload is the point: the fingerprint is taken at the device, so a man-in-the-middle swap between phone and server is detected. This is "capture-time tamper-proofing" and it is genuinely uncommon in competing systems.

### 7. Realtime — the live judgement update

Enable broadcast:

```
alter publication supabase_realtime add table cases;
alter publication supabase_realtime add table documents;
alter publication supabase_realtime add table court_sessions;
alter publication supabase_realtime add table judgements;
```

RLS applies to realtime too — a client only receives rows it is allowed to read. That's worth stating aloud; it's a subtlety most teams get wrong by broadcasting everything and filtering on the client.

Flutter listener on the case detail screen:

```
final channel = supabase
  .channel('case:$caseId')
  .onPostgresChanges(
    event: PostgresChangeEvent.insert,
    schema: 'public',
    table: 'judgements',
    filter: PostgresChangeFilter(
      type: PostgresChangeFilterType.eq,
      column: 'case_id',
      value: caseId,
    ),
    callback: (payload) {
      final j = payload.newRecord;
      setState(() => _judgement = Judgement.fromMap(j));
      _showBanner('Judgement pronounced — ${j['verdict']} · '
        '${DateFormat('dd MMM yyyy, HH:mm').format(
          DateTime.parse(j['pronounced_at']).toLocal())}');
    },
  )
  .onPostgresChanges(
    event: PostgresChangeEvent.insert,
    schema: 'public',
    table: 'court_sessions',
    filter: PostgresChangeFilter(
      type: PostgresChangeFilterType.eq,
      column: 'case_id', value: caseId),
    callback: (p) => _appendSession(CourtSession.fromMap(p.newRecord)),
  )
  .subscribe();
```

On the police officer's device, the case timeline reorders itself and the status chip flips from *In Trial* to *Disposed* with no refresh. That's the moment that sells the demo.

### 8. AI layer (designed in Google AI Studio, called from Edge Functions)

Prototype the prompts in AI Studio, then export the call into a Deno Edge Function so the API key never ships inside the APK.

| Capability                                 | Model call                         | Output into        |
|--------------------------------------------|------------------------------------|--------------------|
| OCR of scanned FIR / handwritten statement | Gemini vision, image + instruction | documents.ocr_text |
| Auto-classification into the 8 doc types   | Gemini, JSON-only response         | documents.doc_type |

|                                                       |                                   |                               |
|-------------------------------------------------------|-----------------------------------|-------------------------------|
| Entity extraction (names, sections, dates, locations) | Gemini structured output          | documents.ai_entities (jsonb) |
| Case summary for a judge opening the bundle           | Gemini over concatenated OCR text | documents.ai_summary          |
| Semantic search across a case                         | Gemini embeddings + pgvector      | search index                  |
| Compliance pre-check on charge sheet                  | Gemini against a checklist prompt | warning banner before submit  |

Classification prompt pattern — always force JSON and always keep a confidence:

```
You are classifying an Indian legal/investigation document.
Return ONLY valid JSON, no markdown fences, no commentary.
Schema:
{"doc_type": one of ["fir", "police_report", "investigation_record",
"witness_statement", "charge_sheet", "court_filing", "evidence_record",
"forensic_report", "legal_notice", "judgement", "other"],
"confidence": 0.0-1.0,
"entities": {"persons": [], "sections": [], "dates": [], "locations": []}}
Document text: <<<{{ocr_text}}>>>
```

**Guardrail to state explicitly:** AI output is *advisory metadata only*. It never alters document content, never decides access, and low-confidence classifications are flagged for human confirmation. An AI that could silently retag a charge sheet as "other" would be a security hole. Saying this unprompted signals maturity.

The **compliance pre-check** is the most differentiating AI feature: before an IO submits a charge sheet, the model checks it against a statutory completeness checklist (required sections cited, witness list attached, seizure memo referenced, timeline present) and warns about gaps. Existing systems catch these problems weeks later at the court registry. That is a concrete, defensible improvement over CCTNS-style workflows.

9. Flutter app structure

```
lib/
├─ main.dart
├─ core/
│   ├── supabase_client.dart
│   ├── theme.dart           // govt-appropriate: high contrast, large tap targets
│   ├── crypto.dart          // sha256 of file bytes
│   └─ audit.dart            // logAction() helper on every screen
├─ auth/
│   ├── register_screen.dart // badge, designation, org picker, role request
│   ├── otp_screen.dart      // 6-box OTP, resend timer
│   ├── pending_approval_screen.dart
│   └─ step_up_dialog.dart    // re-OTP before sensitive actions
├─ shell/
│   └─ role_router.dart       // routes to the right dashboard by role
├─ police/
│   ├── file_case_screen.dart
│   ├── my_cases_screen.dart
│   └─ upload_document_screen.dart
├─ court/
│   ├── cause_list_screen.dart
│   ├── record_session_screen.dart
│   └─ pronounce_judgement_screen.dart
├─ forensic/
│   └─ upload_report_screen.dart
├─ admin/
│   ├── pending_users_screen.dart
│   └─ audit_viewer_screen.dart
└─ shared/
    ├── case_timeline.dart    // unified chronological view
    ├── document_viewer.dart  // watermarked with viewer name + timestamp
    └─ integrity_badge.dart    // green/red hash verification chip
```

**Case timeline** — a single vertical chronology merging FIR, every document upload, every court session, and the judgement, each stamped with actor and exact time. This directly answers "poor auditability" from the problem statement, visually.

**Integrity badge** — on every document, a chip that recomputes the hash on open and shows  *Verified · anchored 14:32* or  *Integrity failure*. Tapping it shows the hash, the anchor transaction ID, and the chain of custody.

**Dynamic watermarking** — when a document is viewed, overlay the viewer's name, badge number and timestamp diagonally across it. If a photo of a confidential witness statement leaks, it identifies who leaked it. Cheap to build, extremely persuasive to judges, and directly addresses "unauthorized access to confidential information."

10. Build order for a hackathon timeline

| Phase | Build                                                  | Hours |
|-------|--------------------------------------------------------|-------|
| 1     | Supabase project, run full schema + RLS, seed 3 orgs   | 3     |
| 2     | Register → OTP → pending → admin approve → login       | 6     |
| 3     | Police: file case, list cases, case detail             | 5     |
| 4     | Document upload with client-side hash + storage policy | 6     |
| 5     | Court: record session, pronounce judgement             | 5     |
| 6     | Realtime subscriptions + timeline UI                   | 4     |
| 7     | Audit log + chain verification screen                  | 4     |
| 8     | Gemini OCR + classification Edge Function              | 5     |
| 9     | Watermarking, integrity badge, polish                  | 4     |
| 10    | Blockchain anchoring function                          | 4     |
| 11    | Rehearse demo on real devices, real wifi               | 3     |

If you run short, cut in this order: blockchain anchoring (keep the hash chain — it's the substance), then semantic search, then compliance pre-check. **Never cut:** RLS, realtime, audit log. Those three are what make it real.

11. Live demo script for the judging round

Five team members, five phones (or four phones + one laptop browser), all on the same live database. Target: 7 minutes.

Setup before you present

- Seed exactly two organisations: *PS Hazratganj, Lucknow* and *District Court, Lucknow*. Nothing else.
- Have the admin account pre-approved. **Every other account is registered live, on stage.**
- Open a second screen showing the Supabase table editor, live.

| # | Who               | Action                                                                                                            | What the judges see                                                                     |
|---|-------------------|-------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| 1 | Member A (Admin)  | Already logged in, shows empty <code>profiles</code> table                                                        | The database is genuinely empty. No demo data.                                          |
| 2 | Member B (Police) | Registers on stage: badge no, SI, PS Hazratganj → receives real OTP → verifies → lands on "Awaiting verification" | Real auth, real OTP, and impersonation is blocked by design                             |
| 3 | Member A          | Approves B. Row flips to <code>approved</code> in the live table view                                             | Departmental verification is a real gate                                                |
| 4 | Member B          | Files <b>CASE-2026-000001</b> , uploads a photographed FIR                                                        | Gemini OCRs it, auto-classifies as <code>fir</code> , extracts sections and names. Hash |

shown.

|    |                  |                                                                                                          |                                                                                                             |
|----|------------------|----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| 5  | Member C (Judge) | Registers + gets approved, is added as participant to the case                                           | Case-scoped access, not blanket role access                                                                 |
| 6  | Judge test       | Ask a judge to take Member D's phone (forensic expert, not on the case) and try to open CASE-2026-000001 | <b>Server returns nothing.</b> Not a hidden screen — an empty result set. This is the moment that wins it.  |
| 7  | Member C         | Records a court session, then pronounces judgement (step-up OTP required)                                | Legally consequential action requires fresh authentication                                                  |
| 8  | All phones       | Member B's screen updates itself                                                                         | Case stage → <i>Disposed</i> , judgement appears with exact timestamp, all documents auto-lock. No refresh. |
| 9  | Member E         | Opens audit trail, runs <code>verify_audit_chain()</code>                                                | Returns zero rows = chain intact                                                                            |
| 10 | Tamper demo      | From the Supabase dashboard, try to update a document's hash                                             | Trigger raises " <i>Document content is immutable</i> ". Then delete an audit row → rule blocks it.         |

Close with one line: "*Everything you just saw was written to a live database under policies the server enforces. Nothing on these screens was hardcoded.*"

Fallbacks — rehearse these

- **Venue wifi dies:** phone hotspot ready, and one team member's device already logged in with the session cached so you can at least show reads.
- **OTP email delayed:** have one pre-registered backup account per role. Say plainly you're switching to it; don't stall silently.
- **Gemini call times out:** the upload must still succeed with OCR marked *pending*. Build this failure path — never let an AI timeout block a document upload. Judges notice graceful degradation.

12. Requirement traceability

| Problem statement requirement            | Where it's satisfied                                                                    |
|------------------------------------------|-----------------------------------------------------------------------------------------|
| Digitize and centralize document storage | Gemini OCR pipeline + Supabase Storage, all 8 doc types as first-class enum             |
| Ensure secure access and confidentiality | RLS + case-participant ABAC + step-up OTP + watermarked viewing                         |
| Prevent unauthorized modifications       | Immutability trigger, version chain, auto-lock on judgement                             |
| Complete audit trail                     | Hash-chained <code>audit_log</code> , unmodifiable by rule, Merkle-anchored             |
| Efficient search and retrieval           | Full-text + trigram + Gemini semantic search over OCR text                              |
| Collaboration among stakeholders         | Shared case workspace across police / court / prosecution / forensic with realtime sync |
| Legal and regulatory compliance          | IT Act §65B evidentiary trail, eSign/DSC path, data localization on Indian region infra |
| Difficulty locating documents quickly    | Semantic search + unified case timeline                                                 |
| Document tampering risks                 | Capture-time hashing, chain anchoring, integrity badge                                  |
| Lack of version control                  | Immutable versioned documents with diff-able chain                                      |
| Inefficient collaboration / delays       | Realtime cross-department updates, no file transfer between departments                 |
| Poor auditability                        | Every read logged, including views and downloads                                        |

### 13. Honest weaknesses — prepare these, don't hide them

Judges trust teams that name their own limits.

1. **Key management is the hard unsolved part.** Encryption keys currently live in Supabase's managed KMS. Production would need an HSM under departmental control. Say this before they ask.
2. **OCR accuracy on handwritten Hindi/Urdu FIRs is imperfect.** Mitigated by human confirmation on low-confidence classification, not solved.
3. **We anchor batches, not every event** — so there is a worst-case 15-minute window where a database compromise isn't yet externally provable. It's a deliberate cost/latency trade-off, and configurable.
4. **Legal admissibility ultimately requires certification** under §65B by a designated officer; the system produces the certificate's supporting evidence but does not replace the human certification.

Naming #3 unprompted, with the trade-off reasoning, is usually worth more than any feature you could add in the remaining time.